Kristiania University College

Android App: Exploiting Vulnerabilities and Writing Secure Code

- Reviewing of vulnerabilities
- Exploiting vulnerabilities
- Principle of least exposure
- Security mechanisms
- Slowing-down a reverse engineer

Reviewing of vulnerabilities

- Its important to understand how an attacker uses an **Android App vulnerabilities** to gain access to devices.
- **Finding vulnerabilities** in pre-installed apps. Know more on how to compromise for a **better chance of securing the apps.**
- Each installed App has its own **attack surface** that can be exploited.

- Emphasis is placed on gaining root access to a device- exploiting Apps with powerful **contexts is a priority** for a bug hunter.
- A good example of a powerful permission is the **install capability**.
- This enables to maximize **return on the invested** time.

Kristiania University College

Reviewing …

Common Vulnerabilities:

- Code injection
- Logic flaws
- Insecure storage
- Application configuration
- Insecure communication
- Logging

Exploiting vulnerabilities

- It should be clear that many **vulnerabilities (remote and local)** can be discovered and exploited on Android devices.
- A **remote exploit** allows an attacker to gain a foothold on the target device.
- Access can occur through a multitude of attack vectors such as **software exploits, man-in-the-middle attacks, or malware**.
- A **local exploit** attempt to escalate the privileges of the malicious code or perform an action on an App that was not intended.

Exploiting vulnerabilities …

Example of tools:



- **Ettercap** is de facto standard for **performing man-in-the-middle** attacks on a network-**ARP poisoning, DNS spoofing, etc.**
- **Drozer** is a framework to exploit Apps- provides a for **sharing of exploits** through a central online repository.
- Impact of the **privilege levels-** Install Package, ADB Shell Access, System User Access, Root User Access, etc.

Exploiting vulnerabilities …

Infiltrating audio:

- Infiltrate using **drozer modules** and **record from the microphone** of the device you have **compromised**.
- The **requirements are** that you have **compromised an App** with the RECORD_AUDIO permission and have retained Context.

Infiltrating SMS

- Infiltrate two-factor authentication that uses OTP (one-time password) tokens. **dz> run post.sms.read -f OTP**
- Send SMS: **dz> run post.sms.send 0745678*** "My message text"**

Exploiting vulnerabilities …

Infiltrating Contacts

- Infiltrate contacts, for example of searching by surname:

  dz> run post.contacts.read -f Derek →| Derek | 0745678* |**

Infiltrating GPS location

- Determine a **user's rough location** even without Wi-Fi hotspot proximity markers can- use **post.capture.location**.

Other infiltrations

- Capturing a user’s screen, stealing files from SD card, cracking patterns, PINs, and passwords

Principle of least exposure

Minimizing Attack Surface

- **The fewer entry points** there are into an App, the smaller the attack surface is.
- Find every **single portion of the App code** that is exposed in some way **to input** from outside sources and **remove it**.
- **If an entry point has to be exposed**, perform security checks **at the entry points** before running other code.

Principle …

Application components

- Apps **should reduce exported** components down to essential.
- Safest case is to export the **main activity** only.

```
<provider
  android:authorities="com.example.app.Provider"
  android:name="com.example.app.Provider"
  android:exported="false" />
```

```
dz> run app.package.attacksurface com.myapp.secure

Attack Surface:

  1 activities exported

  0 broadcast receivers exported

  0 content providers exported

  0 services exported
```

Principle …

Data storage
- Avoid **storing unnecessary** data- such as passwords!
- **Private directory** is protected by the sandboxing
- **SD card** is less protected

Untrusted sources
- Inputs from **SD card**, Wi-Fi, Bluetooth, etc.
- **Verify** authenticity with signature, encryption, etc.
- Be **careful loading** classes or running executables from untrusted locations
- **Cryptographic** protections are the best

Minimal permissions
- Request the **fewest permissions** needed for your app
- It is safer, and avoids worrying users
- Avoid risky permissions like:

  -sharing powerful users (android.uid.system)
  -INSTALL_PACKAGES

Bundling files in the APK
- APK can contain **extra files** by accident
- May contain SSH credentials or other secrets

Security mechanisms

Review entry points

- Trace functions that receive code **from other apps** inside each exported component over the IPC sandbox
- Enforce permissions by setting the **android: permission** attribute in manifest

Permission protection

- **Limit** permission in exported components .. only to apps with same **signature**
- Great care is required if you really want to offer a component for **public use**

COMPONENT	METHOD
Activity	`onCreate()`
Broadcast Receiver	`onReceive()`
Content Provider	`query() insert() update() delete()openFile()`
Service	`onStartCommand()onBind()`

```
<uses-permission
    android:name="android.permission.CAMERA"/>
```

Dangerous permissions:
READ_CALENDAR, WRITE_CALENDAR, **CAMERA,** READ_CALL_LOG, WRITE_CALL_LOG, READ_CONTACTS, WRITE_CONTACTS, GET_ACCOUNTS, ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, SEND_SMS, RECEIVE_SMS

Security mechanisms …

Securing **Activities**:

Task **manager snooping**

- Remove your app from the **recent app list**- to avoid exposing private information on that image
- Put this code in **OnCreate()** to show a blank screen in the list

```
getWindow().addFlags(WindowManager.LayoutParams.FLAG_SECURE;
```

- Set this attribute in an **activity** to remove it entirely from the list

```
intent.addFlags(Intent.FLAG_ACTIVITY_EXCLUDE_FROM_RECENTS);
```

Tapjacking

- Prevent touches from being sent through elements with this attribute:

  android:filterTouchesWhenObscured="true"

- Or by using this method:

  view.setFilterTouchesWhenObscured(true);

Secure trust boundaries

- Make sure there's noway to open an **authenticated activity** from unauthenticated areas of the app
- One way: implement an **app-wide authentication variable**

Browsable Activities

- Can be used directly from a web browser
- High-value targets for attackers- avoid using **BROWSABLE**
- If you use it, consider all possible intents that could cause actions in your app

Other mechanisms

- Disable additions to the dictionary to keep secrets out and mask password displays.
- Add this attribute to an **EditText** box:

```
android:inputType="textVisiblePassword"
android:inputType="textPassword"
```

Securing Android activities

Security mechanisms …

Securing **Content Providers**:

Default export behavior

- Prior to API 17, content providers were **exported by default**
- To prevent this, put this code in the manifest:

```
<provider android:name=".ContentProvider“
android:authorities="com.myapp.ContentProvider"
android:exported="false" >
</provider>
```

SQL injection

- Avoid direct user input inside SQL statement in rawQuery() SQLite method by concatenating queries- use **prepared statements** instead:

```
String[] userInput = new String[] {"book", "wiley"};
Cursor c = database.rawQuery("SELECT * FROM
Products WHERE type=? AND brand=?", userInput);
```

Directory traversal

- The **getCanonicalPath**() method removes .. characters and provides the **absolute** path to a file
- This code uses traverses to limit paths to the /files/ subdirectory of the app's private data directory

```
@Override
public ParcelFileDescriptor openFile (Uri uri, String mode)
{
    try
    {
        String baseFolder = getContext().getFilesDir().getPath();
        File requestedFile = new File(uri.getPath());

        //Only allow the retrieval of files from the /files/
        //directory in the private data directory
        if (requestedFile.getCanonicalPath().startsWith(baseFolder))
            return ParcelFileDescriptor.open(requestedFile,
                    ParcelFileDescriptor.MODE_READ_ONLY);
        else

return null;
    }
    catch (FileNotFoundException e)
    {
        return null;
    }
    catch (IOException e)
    {
        return null;
    }
}
```

Security mechanisms …

Securing **Broadcast Receivers**:

- Secret codes are easily **enumerated** using apps on the Play Store
- An attacker who knows your implemented **secret code** should not be able to have any control over the App.
- Secret codes should be used only for **testing purposes** and must be removed before releasing the application.
- Ensure that the code in the **broadcast receiver** cannot perform an unintended action by invoking the secret code.

```
<receiver android:name=".GpsLocationReceiver">
    <intent-filter>
        <action android:name="android.location.PROVIDERS_CHANGED" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</receiver>
```

```
public class GpsLocationReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        if (intent.getAction().matches("android.location.PROVIDERS_CHANGED")) {
            Toast.makeText(context, "in android.location.PROVIDERS_CHANGED",
            Toast.LENGTH_SHORT).show();
            Intent pushIntent = new Intent(context, LocalService.class);
            context.startService(pushIntent);
        }
    }
}
```

Security mechanisms …

Storing Files Securely:

Encryption
- Use **AES** for symmetric encryption, avoid ECB
- Use **RSA-2048** for asymmetric encryption
- Password hashing advice- **salting**

Random numbers
- **Random**() produces the same series of numbers each time it's run from the same seed
- Better use **SecureRandom-** Java provides methods to seed it from a source of entropy
- **Key generation-** PBKDF2 uses many rounds of hashing to derive a key from a password & be stored in Keystore

```
SecureRandom sr = new SecureRandom();
KeyGenerator generator = KeyGenerator.getInstance("AES");
generator.init(256, sr);
SecretKey key = generator.generateKey();
```

Creating Files and Folders
- Explicitly **set permissions**

```
FileOutputStream secretFile = openFileOutput("secret",
Context.MODE_PRIVATE);
File newdir = getDir("newdir", Context.MODE_PRIVATE);
```

Exposing files
- To allow specified other apps to see a file
- Those apps need com.myapp.docs.READWITE permission.

```
<provider
    android:name=".DocProvider"
    android:authorities="com.myapp.docs"
    android:exported="true"
    android:permission="com.myapp.docs.READWRITE"
    android:grantUriPermissions="false">
        <grant-uri-permission android:pathPrefix="/document/" />
</provider>
```

Security mechanisms …

Secure **Communication & WebView**:

HTTPS
- HTTPS is much safer than HTTP, but depends on trusted Certificate Authorities (CAs)
- Certificate pinning makes HTTPS even more secure- requiring a specific certificate or CA

Local Communications
- **Transferring** data from one app to another
- Android API is the best method
  - Activities with intent-filters
  - In more recent Android versions- ChooserTargets, Shortcuts, etc.
- Using network sockets or the clipboard is less safe

WebView
- WebView enables to display a Web page in an activity
- Often leads to security problems- if loaded over HTTP, subject to interception and modification
- More info- https://www.checkmarx.com/blog/android-webview-secure-coding-practices/

Expose files securely when sending to other Apps.

```
Uri uri = Uri.parse("content://com.myapp.docs/document/1");
Intent intent = new Intent(Intent.ACTION_VIEW);
intent.setDataAndType(uri, "application/pdf");
intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
intent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION);
startActivity(intent);
```

Security mechanisms …

Configuring the Android Manifest:

Backups and Debugging

- If **android:allowBackup** is **false**, an attacker can't back up files with physical access to the device
- **android:debuggable** allows debugging

API Version Targeting

- **minSdkVersion** should be as large as possible
- Lower values **remove new security fixes**
- Values below 17 **export content** providers by default

Slowing down a reverse engineer

- **Android Apps can easily be decompiled** into readable source code.
- Example, try using decompiler to decompile **Sieve.apk**
- **Tamper detection** can be used by which an App is designed to fail to run if it detects signs of modification of its APK file.
- For example, a reverse engineer may have **modified a manifest** to include **android:debuggable=”true”**.

Slowing down …

- So, security efforts may **only slow down** a reverse engineer by making the analysis task more difficult.
- Use **obfuscators** to make decompiled code less readable and harder to follow.
- Example of obfuscators- ProGuard, DashO, DexGuard, etc.
- In Android studio, install ProGuard plugin, change the minifyEnabled property to true- names of classes, methods and variables get like a, b, c, ab, etc.

```
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
}


buildTypes {
    release {
        minifyEnabled true
        proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
    }
}
```

```
private void l()
{
    setContentView(0x7f030022);
    an = (EditText)findViewById(0x7f0900dd);
    an.setFilters(new InputFilter[] {
        e
    });
    ao = (EditText)findViewById(0x7f0900de);
    ao.setFilters(new InputFilter[] {
        e
    });
    ap = (EditText)findViewById(0x7f0900df);
    ap.setFilters(new InputFilter[] {
        e
    });
    aq = (Button)findViewById(0x7f0900e0);
    aq.setText(getResources().getString(0x7f0b00f5));
    aq.setOnClickListener(new ae(this));
    ar = (ImageView)findViewById(0x7f09004d);
    as = (TextView)findViewById(0x7f090042);
    ar.setOnClickListener(g);
    as.setOnClickListener(g);
}
```

Class activities:

1. Individual participation 1:
   Discussion questions via Mentimeter (Weeks 43)
2. Individual participation 2:
   Questions via Kahoot (Week42/43)
3. Group activity:
   List of activities to be provided
4. Work on “Obligatory assignment 2“

NB: Additional information about the activities yet to be provided in separate document.

Next topics

Security in iOS Applications